STANDARD OPERATING PROCEDURE · FIELD GUIDE SERIES

SOP-EM-001

Deploying Microsoft Defender for Endpoint to macOS with Intune

Permissions, onboarding and the Defender app delivered five ways: Intune admin center, Graph PowerShell, Settings Catalog, local script and Jamf Pro, with verification down to mdatp health.

Endpoint Management | Microsoft Defender | macOS



DOCUMENT ID	SOP-EM-001
VERSION	2.0
EFFECTIVE	October 2026
AUDIENCE	Endpoint administrators, security engineers, Mac administrators
AUTHOR	Richmond Dieujuste, Principal Consultant

Rich Dieu IT Solutions LLC · richdieu.com

Sanitized reference edition — client identifiers removed

Contents

1. Concepts You Need First 3

1.1 MDM on macOS and the user-approved channel 3

1.2 The gates Defender needs 4

1.3 Onboarding package, device channel and dynamic groups 5

1.4 Choosing a method 8

2. Access, Prerequisites and Planning 9

2.1 Level of access required 9

2.2 Prerequisites checklist 10

2.3 Decisions to make up front 11

3. Method A · Intune Admin Center 12

4. Method B · Microsoft Graph PowerShell Template 17

5. Methods C, D and E · Other Ways to Deploy 20

5.1 Method C · Intune Settings Catalog and Endpoint security 20

5.2 Method D · Local script onboarding (small numbers) 20

5.3 Method E · Third-party MDM (Jamf Pro overview) 21

6. Verification, Rollback and Troubleshooting 22

6.1 Three places to verify 22

6.2 Detection test and evidence 23

6.3 Rollback and offboarding 23

6.4 Troubleshooting 24

7. Summary, Glossary and Review 26

7.1 Chapter summary 26

7.2 Glossary 26

LEARNING OBJECTIVES

After working through this procedure you will be able to:

- Explain how MDM, TCC/PPPC, system extensions and the content filter let a security agent run silently.

- Prepare the tenant: Defender–Intune connector, Corporate ownership and a dynamic Mac group.
- Deploy the seven permission profiles, the onboarding profile and the Defender app in Intune.
- Automate the deployment with a Microsoft Graph PowerShell template that writes evidence.
- Choose between custom profiles, Settings Catalog, local script and a third-party MDM.
- Verify with Intune, the Defender portal and mdatp health, and troubleshoot failures.

TIME	DIFFICULTY	MINIMUM ROLE	RISK TO DEVICES
≈ 60 min + up to 2 h check-in	Intermediate	Policy and Profile Manager + Application Manager; Security Administrator	Low (additive)

CHAPTER 1

Concepts You Need First

Installing an antivirus agent on Windows is mostly a matter of running an installer. On macOS it is not, and the difference is the single most important idea in this SOP. Apple deliberately makes it hard for *any* software, including security software, to see other apps' files, inspect network traffic or run invisibly in the background. Each of those powers is locked behind a separate privacy or security gate, and on a normal Mac only the user can open each gate by clicking through a prompt. An enterprise cannot rely on hundreds of users clicking the right buttons, so it uses **mobile device management (MDM)** to open the gates silently, in advance, before the agent arrives.

> **KEY CONCEPT · KEYS BEFORE THE TENANT MOVES IN**
>
> Think of the Defender app as a new tenant moving into a secured office building. The building manager (MDM) must cut the keys first: one for the file room (Full Disk Access), one for the network closet (the content filter), one for the plant room (system extensions) and a badge that registers the tenant with head office (the onboarding profile). If the tenant arrives before the keys are cut, they stand at every locked door asking the receptionist (the user) for permission. That is exactly what users see when profiles land after the app.

1.1 MDM on macOS and the user-approved channel

A Mac becomes "managed" when it installs an **MDM enrollment profile** that trusts your MDM server (here, Microsoft Intune). From then on, Intune can push **configuration profiles**: signed XML property lists with the extension `.mobileconfig`. Apple only lets MDM pre-approve sensitive privacy settings when enrollment is *user-approved* (the user explicitly approved the management profile in System Settings) or when the Mac came through **Automated Device Enrollment** (Apple Business Manager), which is user-approved by definition. A Mac that merely has a profile installed by a script does not qualify. Enrollment itself is covered in the companion SOP on enrolling macOS devices into Intune.

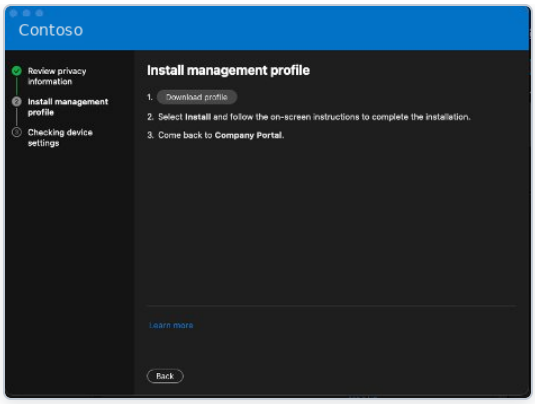


Figure 1. The Company Portal step in which the user downloads and approves the management profile. This approval is what makes the enrollment *user-approved MDM*, a precondition for silently granting Defender its privacy permissions.

1.2 The gates Defender needs

macOS mechanism	What it is	Why Defender needs it
TCC / PPPC	**Transparency, Consent and Control** is the privacy permission database; **Privacy Preferences Policy Control** (PPPC) is the MDM payload that pre-answers it.	Full Disk Access (scanning), Accessibility (DLP), Bluetooth (device control).
System extensions	User-space successors to kernel extensions, approved by team ID and bundle ID.	Endpoint Security (EDR) and network sensors.
Content filter	Network extension allowed to inspect traffic once declared by payload.	Network protection and network EDR.
Background items, notifications	macOS 13+ login-item disclosure and per-app alert permission.	Silent daemon; user-facing threat alerts.

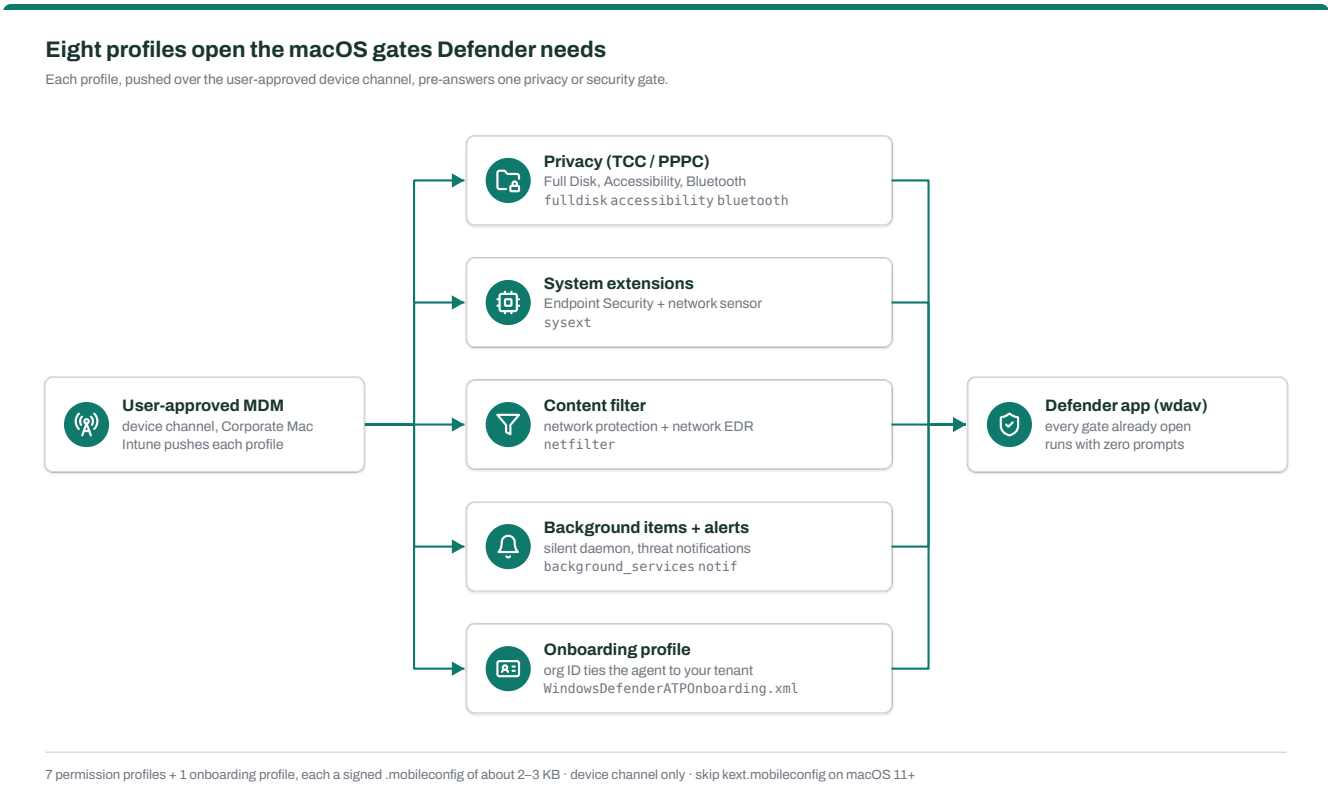


Figure 2. Defender can run silently only because MDM pre-answers each privacy and security gate before the app arrives; the onboarding profile is what ties the agent to your tenant.

1.3 Onboarding package, device channel and dynamic groups

Three more ideas complete the picture.

- **Onboarding package.** The Defender agent is the same for every customer in the world. What ties a particular Mac to *your* tenant is a small, tenant-specific configuration (organization ID and connection settings) that you download from the Microsoft Defender portal and deliver as one more configuration profile. Without it the app installs, runs, and reports to nobody.
- **Device channel versus user channel.** macOS has two MDM channels. *Device channel* profiles install into the system scope and apply to every user of the Mac. *User channel* profiles install for the signed-in user only. System extensions, PPPC and content filters are system-wide settings, so these profiles must always use the **device channel**.
- **Dynamic device group.** Rather than adding Macs to a group by hand, a Microsoft Entra dynamic group evaluates a rule against every device object. The rule `(device.deviceOSType -eq "MacMDM")` collects every Intune-managed Mac automatically, so new Macs are protected without anyone remembering to add them. Dynamic groups require Microsoft Entra ID P1 (included in Microsoft 365 E3/E5 and Business Premium).

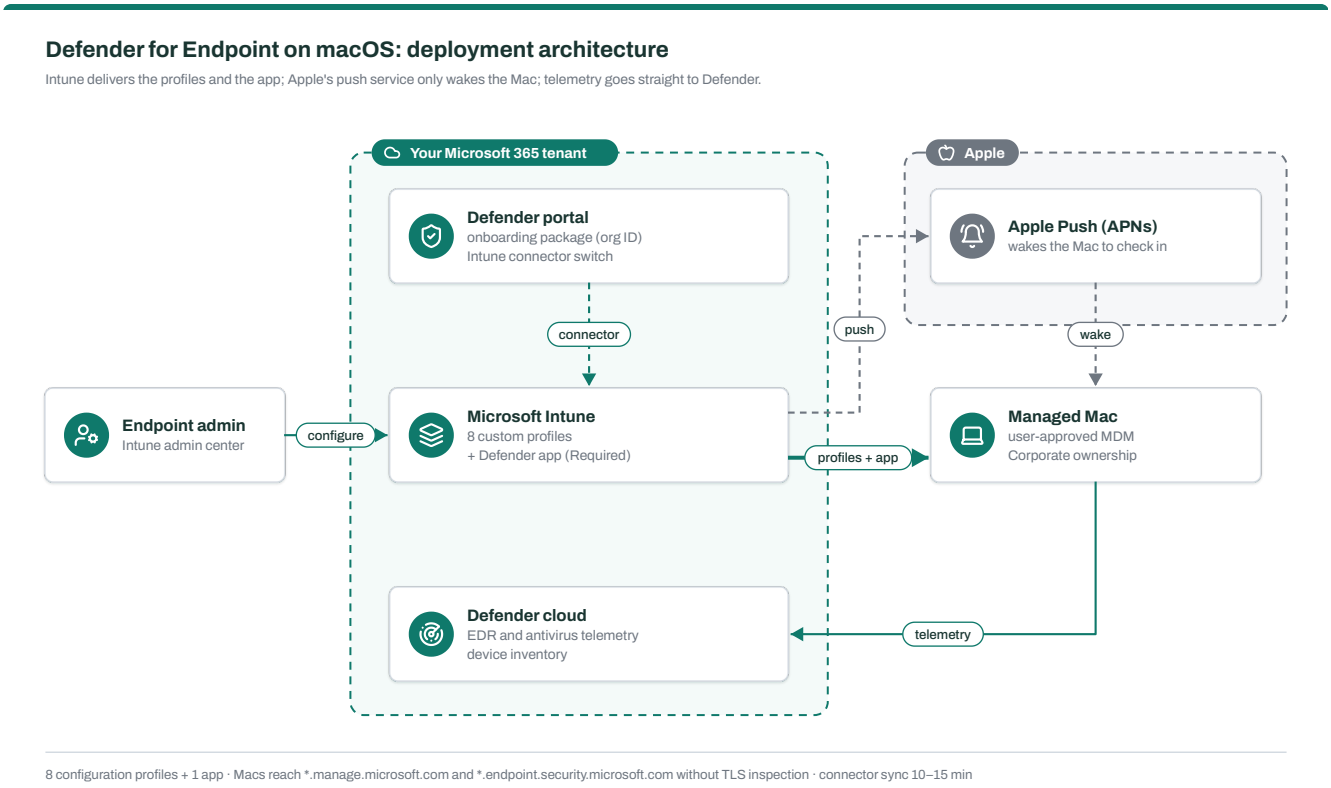


Figure 3. Intune never connects into the Mac: APNs only wakes it, the Mac fetches its profiles and the app, and its telemetry goes straight to the Defender cloud rather than back through Intune.

WHY IT MATTERS · ORDER OF ARRIVAL

Intune does not guarantee the order in which a Mac receives profiles and apps. Creating and assigning all permission profiles *before* the app assignment, as this SOP does, gives them a head start of at least one check-in cycle so that the app normally finds every gate already open.

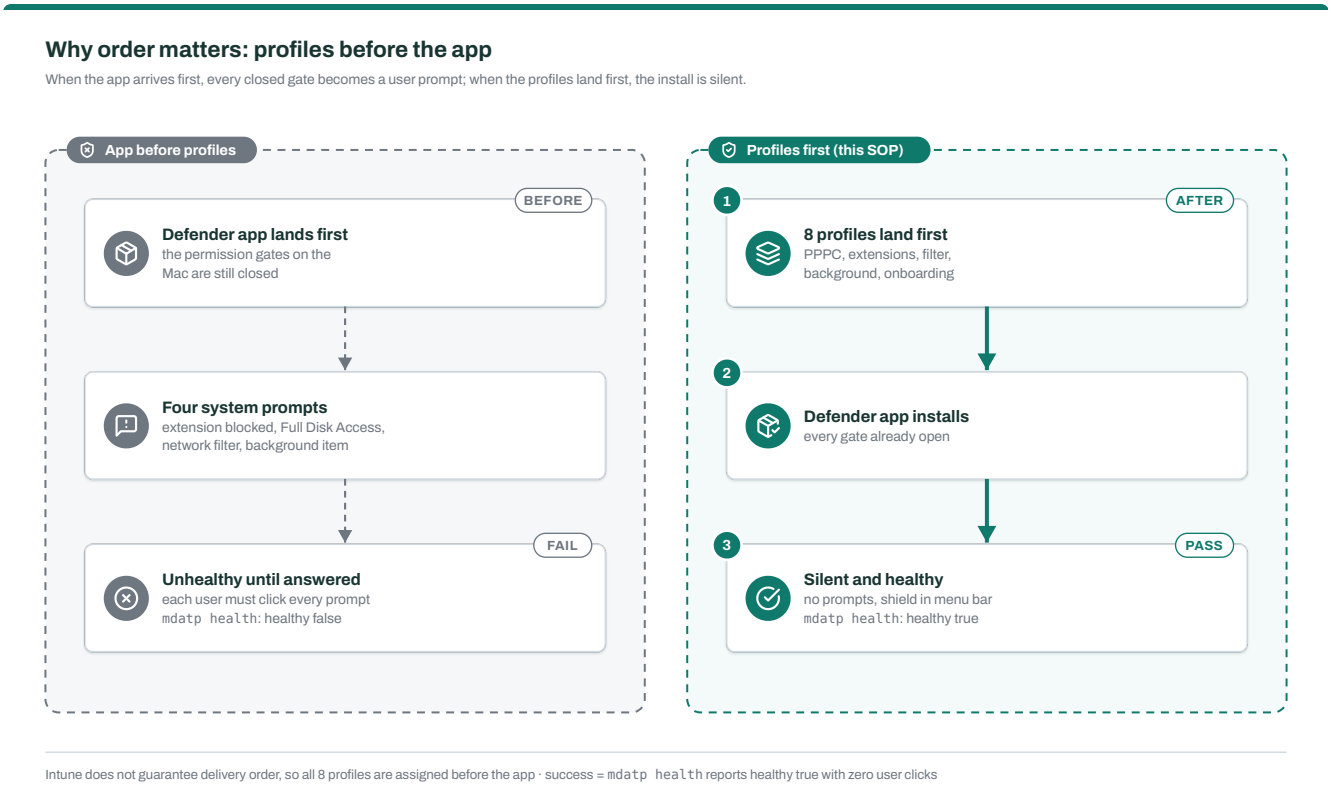


Figure 4. Because Intune does not guarantee delivery order, assigning every profile before the app is what turns four user prompts and an unhealthy agent into a silent, healthy install.

1.4 Choosing a method

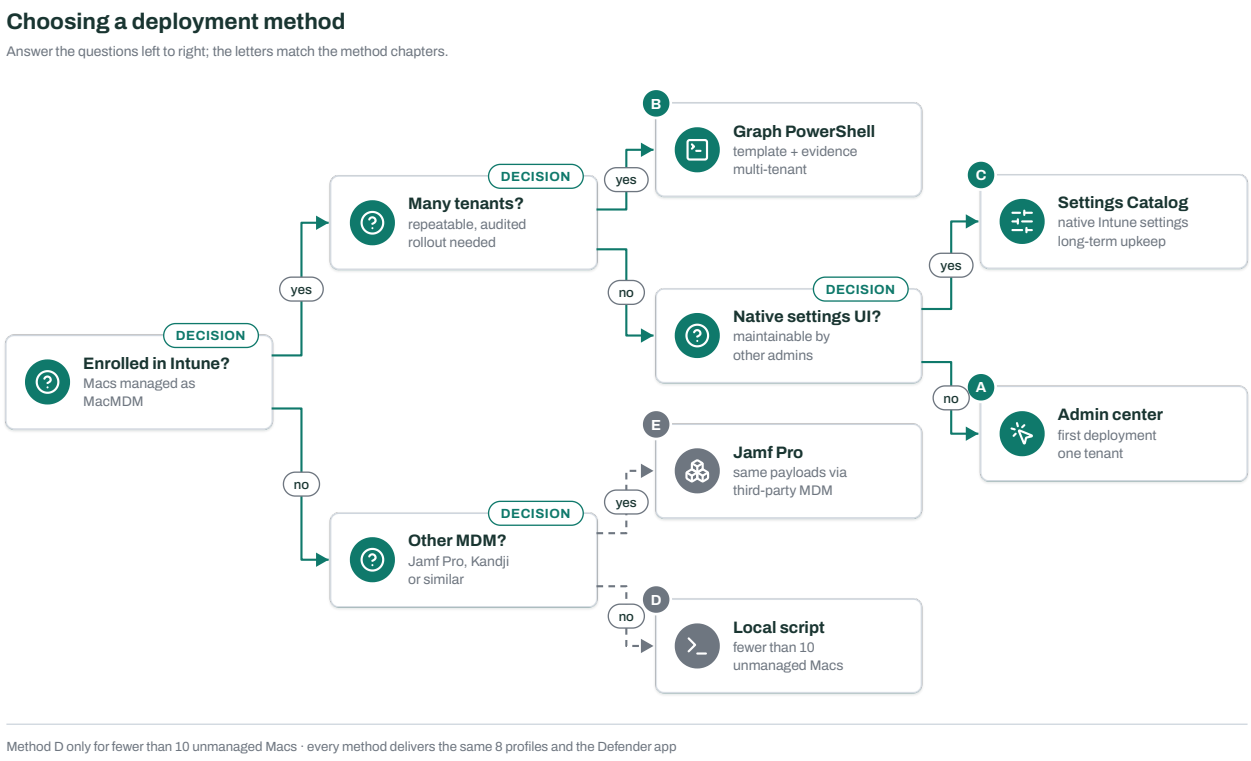


Figure 5. Methods A and B are the managed defaults; C trades upload-once simplicity for long-term readability, and D and E exist only for Macs that Intune does not manage.

A · Intune admin center

Upload Microsoft's signed .mobileconfig files as custom profiles, add the onboarding profile and the built-in Defender app.

BEST FOR: FIRST DEPLOYMENT, ONE TENANT

B · Graph PowerShell

Parameterized template that builds the group, uploads all profiles, adds the app and assigns everything.

BEST FOR: REPEATABLE, MULTI-TENANT, AUDITED

C · Settings Catalog

Native Intune settings (system extensions, PPPC, content filter) plus the Endpoint security antivirus profile.

BEST FOR: LONG-TERM MAINTAINABILITY

D · Local script

Run the installer and a local onboarding script on the Mac; the user approves permissions.

BEST FOR: FEWER THAN 10 UNMANAGED MACS

E · Jamf Pro

Same payloads delivered by a third-party MDM using the Jamf onboarding plist.

BEST FOR: MAC FLEETS ALREADY IN JAMF

CHAPTER 2

Access, Prerequisites and Planning

2.1 Level of access required

The deployment touches three role systems: **Microsoft Entra ID** (groups, Graph consent), **Intune RBAC** (profiles, apps, device properties) and **Defender XDR** (connector, onboarding package). Global Administrator would cover everything but is far more power than needed. The admin account also needs Intune and Defender *licenses*, or the Defender portal reports "A Microsoft Intune license was not found".

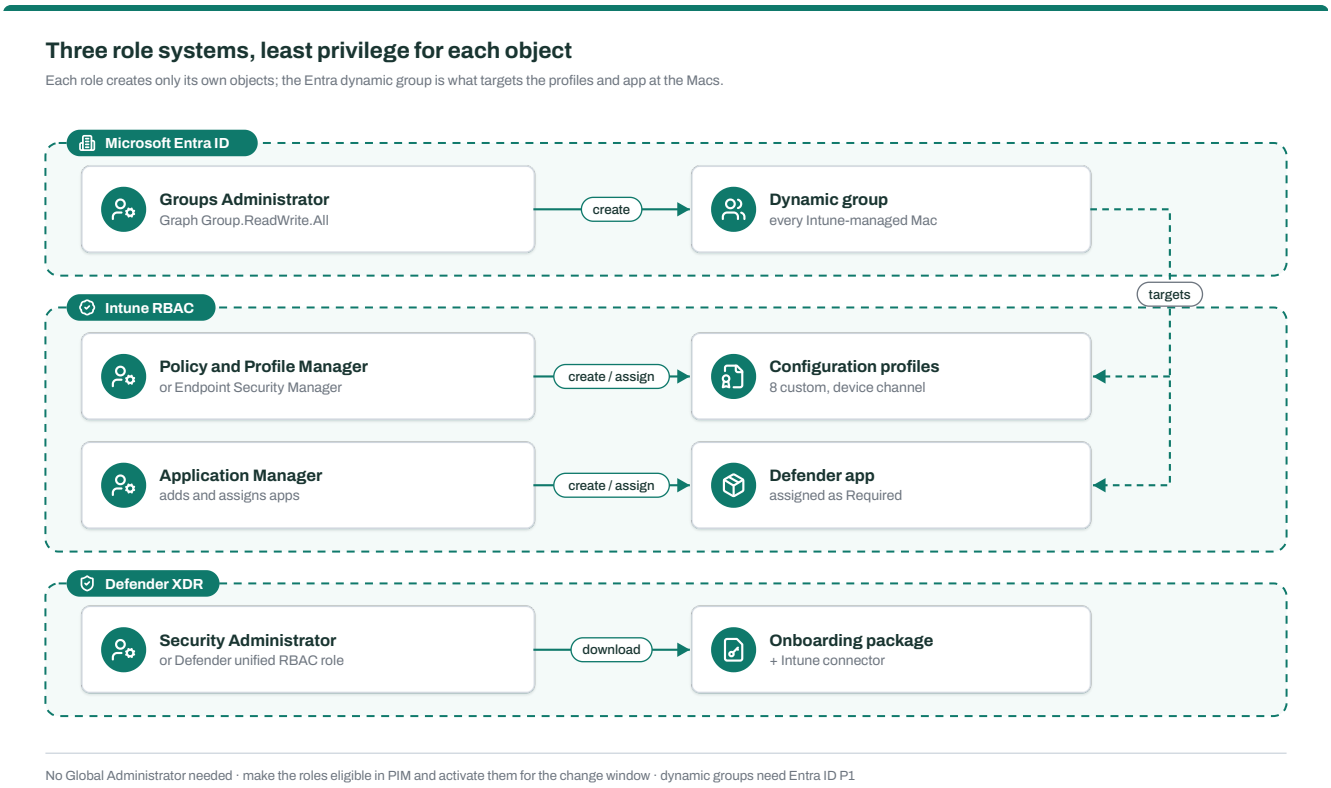


Figure 6. No single role needs tenant-wide power: each role system owns only its own objects, and the Entra dynamic group is the link that targets them at the Macs.

Task	Least-privilege role / permission	Notes
Enable the Intune connection (Defender) and download onboarding/offboarding packages	Security Administrator (Entra), or a Defender unified RBAC role with security settings management	The package is tenant-specific; store it like a secret.

Allow Defender to enforce security configurations (Intune)	Endpoint Security Manager (Intune)	One-time toggle on the connector page.
Create the dynamic device group	Groups Administrator (Entra); Graph `Group.ReadWrite.All`	Requires Entra ID P1.
Set device ownership to Corporate	Custom Intune role with *Managed devices › Read, Update*	Verify the permission in the role editor.
Create and assign configuration profiles	Policy and Profile Manager (Intune); Graph `DeviceManagementConfiguration.ReadWrite.All`	Endpoint Security Manager also works.
Add and assign the Defender app	Application Manager (Intune); Graph `DeviceManagementApps.ReadWrite.All`	Assign as **Required**.
Verify status (portal or script)	Security Reader (Entra); Read Only Operator (Intune); Graph `DeviceManagementManagedDevices.Read.All`	Suitable for a reporting account.
Run `mdatp` on a Mac	Standard user; local admin (`sudo`) for `profiles` and uninstall	Admin consent for Graph scopes is a one-time Cloud Application Administrator task.

> **GOOD PRACTICE · JUST-IN-TIME PRIVILEGE**
>
> Make the Entra roles *eligible* in Privileged Identity Management (PIM) and activate them for the change window. Assign Intune roles to a group whose membership is eligible through *PIM for Groups* for the same time-bound behavior.

2.2 Prerequisites checklist

- [ ] Licensing: Intune Plan 1 and Defender for Endpoint P1/P2 (or Defender for Business), including on the admin account.
- [ ] Macs on a supported macOS release (current and two previous majors), SIP enabled, enrolled as *MacMDM* with user-approved or ADE enrollment.
- [ ] Macs can reach `*.endpoint.security.microsoft.com` and `*.manage.microsoft.com` without TLS inspection.
- [ ] Approved change record (e.g. `CHG0012345`), a pilot group of 2–5 Macs, and a plan for any existing third-party antivirus.

2.3 Decisions to make up front

Decision	Guidance
Pilot versus all Macs	Assign first to a static pilot group; switch to the dynamic group after 48 hours without issues.
Device ownership	Corporate. Personal Macs may prompt for or refuse some payloads.
Connectivity type	*Streamlined* (consolidated domain set), recommended for new deployments.
Naming	`macOS - Defender - <payload>` in Intune; `Defender - <payload>` as seen by the user.

CHAPTER 3

Method A · Intune Admin Center

This is Microsoft's documented path and the best way to learn what each piece does. It is split into five stages that mirror the deployment order.

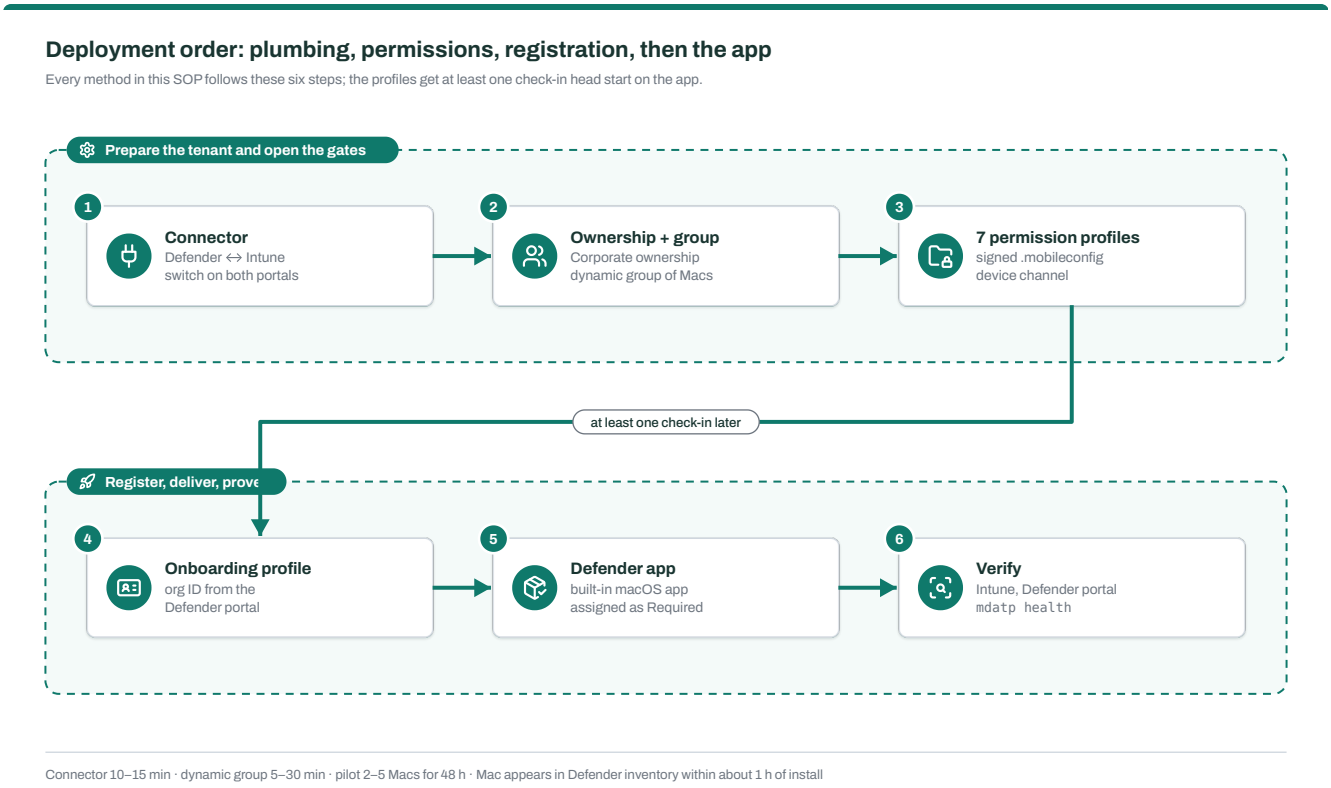


Figure 7. Tenant plumbing and permission profiles always come before the onboarding profile and the app, so every gate is open by the time the agent arrives.

STAGE 1 Connect Defender and Intune

1. Sign in to the Microsoft Defender portal (**security.microsoft.com**) and go to **Settings › Endpoints › General › Advanced features.**
2. Set **Microsoft Intune connection** to **On** and select **Save preferences**. Allow 10–15 minutes.
3. In the Intune admin center (**intune.microsoft.com**) open **Endpoint security › Microsoft Defender for Endpoint.** Confirm *Connection status: Enabled* with a recent *Last synchronized* time.
4. On the same page set **Allow Microsoft Defender for Endpoint to enforce Endpoint Security Configurations** to **On**

and select **Save**.

> **NOTE · GREYED-OUT TOGGLE**
> If the toggle is unavailable or the portal warns that no Intune license was found, the *signed-in admin* lacks an Intune license. Assign one in the Microsoft 365 admin center, sign out and back in, and retry.

STAGE 2 Confirm ownership and build the target group

1. Go to **Devices › All devices**, filter *OS = macOS*. For each Mac confirm *Management type: MacMDM*, a recent *Last check-in* and the expected compliance state.
2. Open a Mac, select **Properties**, set **Device ownership** to **Corporate**, acknowledge and **Save**. (Bulk alternative: add serial numbers as corporate device identifiers before enrollment.)
3. In **Groups › All groups › New group** choose *Group type: Security*, name `macOS Devices`, *Membership type: Dynamic Device*.
4. Select **Add dynamic query**, switch to the rule editor and enter the rule below. **Save**, then **Create**.
5. Wait 5–30 minutes and confirm the pilot Macs appear under **Members**.

DYNAMIC MEMBERSHIP RULE — TEXT

```
(device.deviceOSType -eq "MacMDM")
```

> **WATCH OUT · INVISIBLE SPACES**
> A space inside the quotes (`" MacMDM"`) makes the rule match zero devices without any error. Use **Validate Rules** against a known Mac before saving.

STAGE 3 Create the seven permission profiles

Microsoft publishes signed profiles in the `microsoft/mdatp-xplat` GitHub repository under **macos/mobileconfig/profiles**. Download each file with the **Download raw file** button. Right-click › "Save link as" on the folder listing saves the GitHub *web page*, not the profile, and is the most common cause of profile errors.

File	Grants	Policy name in Intune
fulldisk.mobileconfig	Full Disk Access (TCC) for Defender components	macOS - Defender - Full Disk Access
sysext.mobileconfig	Approves the Endpoint Security and network system extensions	macOS - Defender - System Extensions
netfilter.mobileconfig	Declares the network content filter	macOS - Defender - Network Filter
background_services.mobileconfig	Pre-approves background/login items	macOS - Defender - Background Services
notif.mobileconfig	Allows Defender notifications	macOS - Defender - Notifications
accessibility.mobileconfig	Accessibility access (endpoint DLP)	macOS - Defender - Accessibility
bluetooth.mobileconfig	Bluetooth access (device control)	macOS - Defender - Bluetooth

Skip `kext.mobileconfig` (legacy kernel extensions, not used on Apple silicon or macOS 11+) and, for the first rollout, `sysext_restricted.mobileconfig`.

Sanity-check the downloads: each file should be roughly 2–3 KB and start with `<?xml version="1.0" encoding="UTF-8"?>` followed by a `<plist>`. If you see `<!DOCTYPE html>`, download it again. Then repeat the following for each of the seven files:

1. **Devices › macOS › Configuration › + Create › New policy**. Profile type `Templates` › `Custom` › `Create`.
2. **Basics**: name from the table above and a description that cites the change record.
3. **Configuration settings**: *Custom configuration profile name* `Defender - <payload>` (shown to the user in System Settings); *Deployment channel* **Device channel**; upload the `.mobileconfig` and confirm the XML preview appears.
4. **Assignments**: `+ Add groups` › `macOS Devices` (or the pilot group).
5. **Review + create** › `Create`.

STAGE 4 Deploy the onboarding profile

1. In the Defender portal open **Settings › Endpoints › Device management › Onboarding.**
2. Select *Operating system: macOS*, *Connectivity type: Streamlined*, *Deployment method: Mobile Device Management / Microsoft Intune*, then **Download onboarding package**.
3. Extract the zip. Despite the word "Windows" in its legacy file name it is the macOS package. Open the `intune` folder and locate `WindowsDefenderATPOnboarding.xml` (about 10 KB). Ignore the `jamf` folder and any `kext.xml`.
4. Create one more custom profile exactly as in Stage 3: Intune name `macOS - Defender - Onboarding`, custom profile name `Defender - Onboarding`, **Device channel**, upload the XML, assign to the same group.
5. The macOS configuration list should now show **eight** Defender policies of type *Custom*.

> **CAUTION · TREAT THE PACKAGE AS SENSITIVE**
>
> The onboarding file contains your organization identifier and lets any device that runs it report into your tenant. Do not attach it to tickets, email it or commit it to a public repository. Store it in a restricted location and delete local copies after upload.

STAGE 5 Add the Defender app as Required

1. **Apps › macOS › + Create**. Under *Microsoft Defender for Endpoint* choose **macOS** and **Select**. Intune fetches the current installer from Microsoft; there is nothing to upload.
2. **App information**: review the pre-filled values and select **Next**.
3. **Assignments**: under **Required** add `macOS Devices`. Leave *Available for enrolled devices* empty and do not use *All users/All devices*.
4. **Review + create** › **Create**.

WHY IT MATTERS · REQUIRED, NOT AVAILABLE

An *Available* assignment only lists the app in Company Portal for users to install if they choose. A security control that depends on user choice is not a control. Security agents are always **Required**.

CHAPTER 4

Method B · Microsoft Graph PowerShell Template

Every object built in Method A is a Microsoft Graph resource: the group is a `group`, each profile is a `macOSCustomConfiguration`, and the app is a `macOSMicrosoftDefenderApp`. Scripting them gives you an identical result in every tenant, a peer-reviewable change plan and an evidence file. The template uses the **Microsoft Graph PowerShell SDK** (`Install-Module Microsoft.Graph -Scope CurrentUser`) with delegated sign-in, so actions are attributed to the operator in the Intune audit log.

DEPLOY-MDEMACOS.PS1 · BASE TEMPLATE POWERSHELL

```powershell
<#
.SYNOPSIS  Deploy Microsoft Defender for Endpoint to macOS through Intune (Microsoft Graph).
           Builds the dynamic group, uploads 7 permission profiles + the onboarding profile
           (device channel), adds the Defender app as Required, writes a JSON evidence file.
.NOTES     Modules: Microsoft.Graph.Authentication, .Groups, .DeviceManagement
           Roles  : Groups Administrator (Entra) + Policy and Profile Manager
                    + Application Manager (Intune). Defender-Intune connector already enabled.
#>
[CmdletBinding(SupportsShouldProcess)]
param(
  [Parameter(Mandatory)] [ValidateScript({ Test-Path $_ -PathType Container })]
  [string] $ProfileFolder,                    # the 7 .mobileconfig files from mdatp-xplat
  [Parameter(Mandatory)] [ValidateScript({ Test-Path $_ -PathType Leaf })]
  [string] $OnboardingXml,                    # intune\WindowsDefenderATPOnboarding.xml
  [Parameter(Mandatory)] [string] $ChangeRef, # e.g. CHG0012345
  [string] $GroupName    = 'macOS Devices',
  [string] $PolicyPrefix = 'macOS - Defender - ',
  [string] $UserPrefix   = 'Defender - ',
  [switch] $SkipApp,                          # profiles only (e.g. app added in portal)
  [string] $EvidencePath = ".\$ChangeRef-mde-macos-evidence.json"
)
$ErrorActionPreference = 'Stop'
$api = 'https://graph.microsoft.com/v1.0'
$json = 'application/json'

# ---- Stage 0: connect with least-privilege delegated scopes ----------------
Connect-MgGraph -NoWelcome -Scopes 'Group.ReadWrite.All',
  'DeviceManagementConfiguration.ReadWrite.All', 'DeviceManagementApps.ReadWrite.All',
  'DeviceManagementManagedDevices.Read.All'
$evidence = [ordered]@{
  ChangeRef = $ChangeRef; Operator = (Get-MgContext).Account
  StartedUtc = (Get-Date).ToUniversalTime().ToString('o'); Profiles = @()
}
function Get-GraphAll([string]$Uri) {          # follows @odata.nextLink paging
  $out = @()
  do { $r = Invoke-MgGraphRequest GET $Uri; $out += $r.value; $Uri = $r.'@odata.nextLink' }
  while ($Uri)
  $out
}
function Get-Target($Id) {
  @{ '@odata.type' = '#microsoft.graph.groupAssignmentTarget'; groupId = $Id }
}

# ---- Stage 1: dynamic device group for all Intune-managed Macs -------------
$group = Get-MgGroup -Filter "displayName eq '$GroupName'" | Select-Object -First 1
```

```
if (-not $group -and $PSCmdlet.ShouldProcess($GroupName, 'Create dynamic group')) {
  $group = New-MgGroup -DisplayName $GroupName -SecurityEnabled -MailEnabled:$false `
    -MailNickname "macos-mde-$(Get-Random -Maximum 99999)" `
    -Description "All macOS devices for MDE deployment ($ChangeRef)" `
    -GroupTypes 'DynamicMembership' -MembershipRuleProcessingState 'On' `
    -MembershipRule '(device.deviceOSType -eq "MacMDM")'
}
$evidence.GroupId = $group.Id

# ---- Stage 2: custom profiles (7 permission payloads + onboarding) ---------
$payloads = [ordered]@{
  'fulldisk.mobileconfig'            = 'Full Disk Access'
  'sysext.mobileconfig'              = 'System Extensions'
  'netfilter.mobileconfig'           = 'Network Filter'
  'background_services.mobileconfig' = 'Background Services'
  'notif.mobileconfig'               = 'Notifications'
  'accessibility.mobileconfig'       = 'Accessibility'
  'bluetooth.mobileconfig'           = 'Bluetooth'
}
$files = foreach ($k in $payloads.Keys) {
  [pscustomobject]@{ Path = Join-Path $ProfileFolder $k; Label = $payloads[$k] }
}
$files += [pscustomobject]@{ Path = $OnboardingXml; Label = 'Onboarding' }

$existing = Get-GraphAll "$api/deviceManagement/deviceConfigurations?`$select=id,displayName"
foreach ($f in $files) {
  $name = "$PolicyPrefix$($f.Label)"
  $text = Get-Content $f.Path -Raw
  if ($text -notmatch '^\s*<\?xml' -or $text -match '<!DOCTYPE html') {
    throw "$($f.Path) is not a plist - re-download the RAW file."
  }
  $cfg = $existing | Where-Object displayName -eq $name | Select-Object -First 1
  if (-not $cfg -and $PSCmdlet.ShouldProcess($name, 'Create macOS custom profile')) {
    $bytes = [IO.File]::ReadAllBytes((Resolve-Path $f.Path))
    $body = @{
      '@odata.type'     = '#microsoft.graph.macOSCustomConfiguration'
      displayName       = $name
      description       = "Defender for Endpoint on macOS - $ChangeRef"
      payloadName       = "$UserPrefix$($f.Label)"   # name shown in System Settings
      payloadFileName   = Split-Path $f.Path -Leaf
      payload           = [Convert]::ToBase64String($bytes)
      deploymentChannel = 'deviceChannel'            # NEVER userChannel here
    } | ConvertTo-Json
    $uri = "$api/deviceManagement/deviceConfigurations"
    $cfg = Invoke-MgGraphRequest POST $uri -Body $body -ContentType $json
  }
  if ($cfg -and $group -and $PSCmdlet.ShouldProcess($name, "Assign to $GroupName")) {
    $a = @{ assignments = @(@{ target = (Get-Target $group.Id) }) } | ConvertTo-Json -Depth 5
    $uri = "$api/deviceManagement/deviceConfigurations/$($cfg.id)/assign"
    Invoke-MgGraphRequest POST $uri -Body $a -ContentType $json | Out-Null
  }
  $evidence.Profiles += [pscustomobject]@{ Name = $name; Id = $cfg.id }
}

# ---- Stage 3: Defender app (Intune fetches the installer), Required --------
$appName = 'Microsoft Defender for Endpoint (macOS)'
if (-not $SkipApp -and $group -and $PSCmdlet.ShouldProcess($appName, 'Add + assign Required')) {
  $body = @{ '@odata.type' = '#microsoft.graph.macOSMicrosoftDefenderApp'
             displayName = $appName; publisher = 'Microsoft'
             description = "Deployed under $ChangeRef" } | ConvertTo-Json
  $app = Invoke-MgGraphRequest POST "$api/deviceAppManagement/mobileApps" -Body $body -ContentType $json
  $a = @{ mobileAppAssignments = @(@{
```

```
          '@odata.type' = '#microsoft.graph.mobileAppAssignment'
          intent = 'required'; target = (Get-Target $group.Id) }) } | ConvertTo-Json -Depth 6
  $uri = "$api/deviceAppManagement/mobileApps/$($app.id)/assign"
  Invoke-MgGraphRequest POST $uri -Body $a -ContentType $json | Out-Null
  $evidence.AppId = $app.id
}

# ---- Stage 4: state snapshot for the change record --------------------------
$evidence.Macs = Get-MgDeviceManagementManagedDevice -Filter "operatingSystem eq 'macOS'" -All |
  Select-Object DeviceName, OSVersion, ManagedDeviceOwnerType, ComplianceState, LastSyncDateTime
$evidence.ProfileStatus = foreach ($p in $evidence.Profiles | Where-Object Id) {
  $uri = "$api/deviceManagement/deviceConfigurations/$($p.Id)/deviceStatusOverview"
  $o = Invoke-MgGraphRequest GET $uri
  [pscustomobject]@{ Profile = $p.Name; Success = $o.successCount; Pending = $o.pendingCount
                     Error = $o.errorCount; Failed = $o.failedCount }
}
$evidence | ConvertTo-Json -Depth 6 | Out-File $EvidencePath -Encoding utf8
Write-Host "Done. Evidence written to $EvidencePath" -ForegroundColor Green
```

EXAMPLE · RUNNING THE TEMPLATE

INVOCATION — POWERSHELL

```
.\Deploy-MdeMacOS.ps1 -ProfileFolder 'C:\MDE-macOS-Profiles' `
  -OnboardingXml 'C:\MDE-macOS-Profiles\intune\WindowsDefenderATPOnboarding.xml' `
  -ChangeRef 'CHG0012345' -GroupName 'macOS Devices - Pilot' -WhatIf     # preview first, then run without -WhatIf
```

Use a static pilot group name first. When the pilot is healthy, run again with `-GroupName 'macOS Devices'` : existing profiles are detected by name and only the new assignment is added.

NOTE · API VERSIONS

The template uses Graph **v1.0**. If your tenant rejects the `deploymentChannel` property or the `macOSMicrosoftDefenderApp` type, switch `$api` to the `beta` endpoint for that call, or add the app in the admin center (Method A, Stage 5) and run the template with `-SkipApp` . Beta endpoints can change without notice and are not supported for production automation.

KEY CONCEPT · WHY BASE64?

A `.mobileconfig` is an XML file, but Graph transports it as an opaque binary blob in the `payload` property. Encoding it as base64 guarantees Intune stores the exact bytes Microsoft signed, which matters because signed profiles fail validation on the Mac if a single character changes.

CHAPTER 5

Methods C, D and E · Other Ways to Deploy

5.1 Method C · Intune Settings Catalog and Endpoint security

The Settings Catalog exposes Apple's payloads as native Intune settings, so instead of uploading opaque XML you configure (and later read and diff) each value in the portal. Use it when you want the policy to be maintainable by people who did not build it, or to merge Defender's requirements with other apps' PPPC and extension settings. The onboarding payload is still delivered as a custom profile.

1. **Devices › macOS › Configuration › + Create › New policy** › profile type **Settings catalog**.
2. **System extensions** (search *System Extensions*): allow team identifier `UBF8T346G9` (Microsoft) with bundle IDs `com.microsoft.wdav.epsext` and `com.microsoft.wdav.netext`.
3. **Privacy Preferences Policy Control**: grant *SystemPolicyAllFiles* (Full Disk Access) to `com.microsoft.wdav`, `com.microsoft.wdav.epsext` and the other identifiers listed in Microsoft's `fulldisk.mobileconfig`, each with its code requirement copied from that file.
4. **Content filter**: configure a plugin-type web content filter that matches the values in `netfilter.mobileconfig` (filter data provider bundle ID `com.microsoft.wdav.netext`).
5. Add **Notifications** and **Managed login items / background items** for `com.microsoft.wdav`, then assign to the device group.
6. For Defender's own preferences (real-time protection, cloud protection, exclusions, PUA) use **Endpoint security › Antivirus › + Create policy › macOS › Microsoft Defender Antivirus** rather than a hand-written `com.microsoft.wdav` plist.

> **GOOD PRACTICE · ONE SOURCE OF TRUTH PER PAYLOAD**
>
> Never deliver the same payload type for the same app from both a custom profile and the Settings Catalog. Conflicting PPPC or system extension payloads produce unpredictable results. When you migrate to Method C, unassign the matching custom profile in the same change.

Settings Catalog category names change as Apple adds payloads. Copy code requirements and identifiers from Microsoft's current `mdatp-xplat` profiles rather than from memory.

5.2 Method D · Local script onboarding (small numbers)

For a handful of Macs that are not managed by any MDM (a lab, a contractor's loaner), Defender can be installed and onboarded locally. Because no MDM pre-approves anything,

the user (or an administrator at the keyboard) must approve every gate manually, which is why Microsoft limits this method to small numbers.

1. In the Defender onboarding page choose *macOS* and *Deployment method: Local Script*. Download the installer package (`wdav.pkg`) and the onboarding package.
2. On the Mac, install the app and run the onboarding script with administrator rights (commands below; the script name can vary by package version).
3. Approve the system extensions in **System Settings › General › Login Items & Extensions** (macOS 15+) or **Privacy & Security** (earlier releases).
4. Grant Full Disk Access to Microsoft Defender and its security extension in **System Settings › Privacy & Security › Full Disk Access**. Allow notifications when prompted.

TERMINAL ON THE MAC (ADMIN) BASH

```
sudo installer -pkg ~/Downloads/wdav.pkg -target /
sudo bash ~/Downloads/MicrosoftDefenderATPOnboardingMacOs.sh
mdatp health --field healthy          # expect: true (after permissions are approved)
```

5.3 Method E · Third-party MDM (Jamf Pro overview)

If Macs are managed by Jamf Pro (or Kandji, Mosyle and similar), the concepts are identical; only the console differs. Download the onboarding package with *Deployment method: Mobile Device Management / Microsoft Intune* and use the `jamf` folder, which contains `WindowsDefenderATPOnboarding.plist`.

Requirement	Jamf Pro implementation
Onboarding	*Application & Custom Settings* › upload, preference domain `com.microsoft.wdav.atp`.
Permissions	*Privacy Preferences Policy Control*, *System Extensions* (team ID `UBF8T346G9`), *Content Filter*, *Notifications* and *Managed Login Items* payloads.
Defender preferences	*Application & Custom Settings*, domain `com.microsoft.wdav` (Microsoft publishes a Jamf JSON schema).
App	Upload `wdav.pkg`; install with a policy scoped to the smart group, after the profiles.

Defender works with Jamf, but Intune-specific features (the Defender–Intune connector, Intune compliance using Defender risk score) require the device to be Intune-managed or use Jamf's own Intune partner compliance integration.

CHAPTER 6

Verification, Rollback and Troubleshooting

Allow 30 minutes to 2 hours after the app assignment. Macs check in with Intune roughly every 8 hours on their own; a user can force a check-in from **Company Portal › Devices › <this Mac> › Check status**. Verify in three places, because each one proves something different.

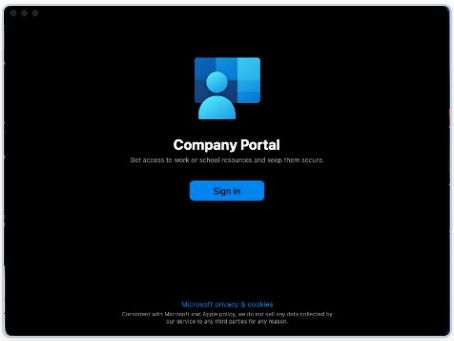


Figure 8. Company Portal on the Mac. After sign-in, the device page offers `Check status`, which triggers an immediate MDM check-in.

6.1 Three places to verify

In Intune: were the objects delivered?

- ☐ **Apps › macOS › Microsoft Defender for Endpoint (macOS) › Overview**: target Macs counted as *Installed*; `Device install status` lists each one.
- ☐ **Devices › All devices › <Mac> › Device configuration**: all eight Defender profiles show *Succeeded*. *Pending* means wait; *Error* means investigate (see Troubleshooting below).

In the Defender portal: is the Mac reporting?

Open **Assets › Devices** and confirm each Mac appears with *OS platform: macOS*, a recent *Last device update* and onboarding status *Onboarded*. This is the definitive test: a Mac that is "Installed" in Intune but absent here did not receive or apply the onboarding profile. For many Macs, use advanced hunting:

ADVANCED HUNTING · MACOS ONBOARDING STATE KQL

```
DeviceInfo
| where Timestamp > ago(1d) and OSPlatform == "macOS"
| summarize arg_max(Timestamp, *) by DeviceId
| project DeviceName, OSVersion, OnboardingStatus, SensorHealthState, Timestamp
| order by DeviceName asc
```

On the Mac: is the agent healthy?

The `mdatp` command-line tool is installed with Defender and is the fastest way to see what the agent itself believes.

TERMINAL ON THE MAC BASH

```
mdatp health                                  # full report
mdatp health --field healthy                  # true = all components working
mdatp health --field health_issues            # [] when healthy; otherwise names the missing permission
mdatp health --field org_id                   # must equal your tenant's Defender organization ID
mdatp health --field real_time_protection_enabled
mdatp connectivity test                       # checks every Defender service URL from this Mac
systemextensionsctl list                      # epsext and netext should be [activated enabled]
sudo profiles show -type configuration | grep -i "Defender -"   # the 8 profiles by user-facing name
```

Field	Healthy value	If not
`healthy`	`true`	Read `health_issues`.
`licensed`	`true`	Onboarding profile missing or wrong tenant.
`org_id`	Your organization GUID	Empty: onboarding not applied.
`full_disk_access_enabled`	`true`	`fulldisk.mobileconfig` failed.
`real_time_protection_enabled`	`true`	Policy disabled it, or extensions not approved.

6.2 Detection test and evidence

Download the harmless EICAR test file, which every antivirus product detects by design, and confirm Defender quarantines it and that an alert appears in the Defender portal within a few minutes:

EICAR TEST BASH

```
curl -o ~/Downloads/eicar.com.txt https://secure.eicar.org/eicar.com.txt
mdatp threat list                             # the EICAR detection should be listed
```

For the change record attach: the evidence JSON from Method B (or screenshots of the eight profiles and app status), an export of the Defender device inventory filtered to macOS, the output of `mdatp health` from one pilot Mac, and the EICAR alert ID.

6.3 Rollback and offboarding

Situation	Action
Pilot shows problems	Remove the pilot group from the app assignment first, then from the profiles. Removing profiles while the app is still installed causes permission prompts and an unhealthy agent.

Permanently remove Defender from a Mac	Deploy the macOS **offboarding** package (Defender portal, Settings › Endpoints › Offboarding) as a custom profile, then assign the app as *Uninstall*, or run `sudo '/Library/Application Support/Microsoft/Defender/uninstall/uninstall'`. Remove the profiles last.
Tenant migration	The onboarding package is tenant-specific and does not expire. Offboard every Mac from the old tenant and onboard with the new package.
Profile refresh	Microsoft updates the `mdatp-xplat` profiles occasionally. Review quarterly; replace the file in the existing Intune policy (do not create a duplicate).

6.4 Troubleshooting

Symptom	Likely cause and fix
Intune connector shows *Unresponsive*	Toggle the Microsoft Intune connection off, save, wait 5 minutes, toggle on, save, wait 15 minutes.
"A Microsoft Intune license was not found"	The admin account lacks an Intune license. Assign one and sign in again.
Dynamic group has 0 members after 30 minutes	Spaces or wrong case in the rule, or Macs not MDM-enrolled. Validate the rule against a known device.
Profile shows *Error*	File saved as HTML (re-download the raw file), or deployment channel set to User. Open the profile's Device status for the exact error.
App installed but Mac missing from Defender	Onboarding profile not assigned or not applied. Check `mdatp health --field org_id`.
User prompted to allow a system extension or filter	The app arrived before `sysext`/`netfilter` profiles, or the enrollment is not user-approved, or the Mac is Personal. Let the user allow, then fix ownership and ordering for the next Mac.
`health_issues` lists "no active event provider" or full disk access	PPPC profile missing or a duplicate conflicting PPPC payload exists. Remove duplicates and re-sync.
Shield icon missing from the menu bar	App installed but not yet launched, or blocked as a login item. Restart; check System Settings › General › Login Items & Extensions.
Deployment takes hours	Normal check-in cycle. Use Company Portal Check status or the Intune Sync device action.

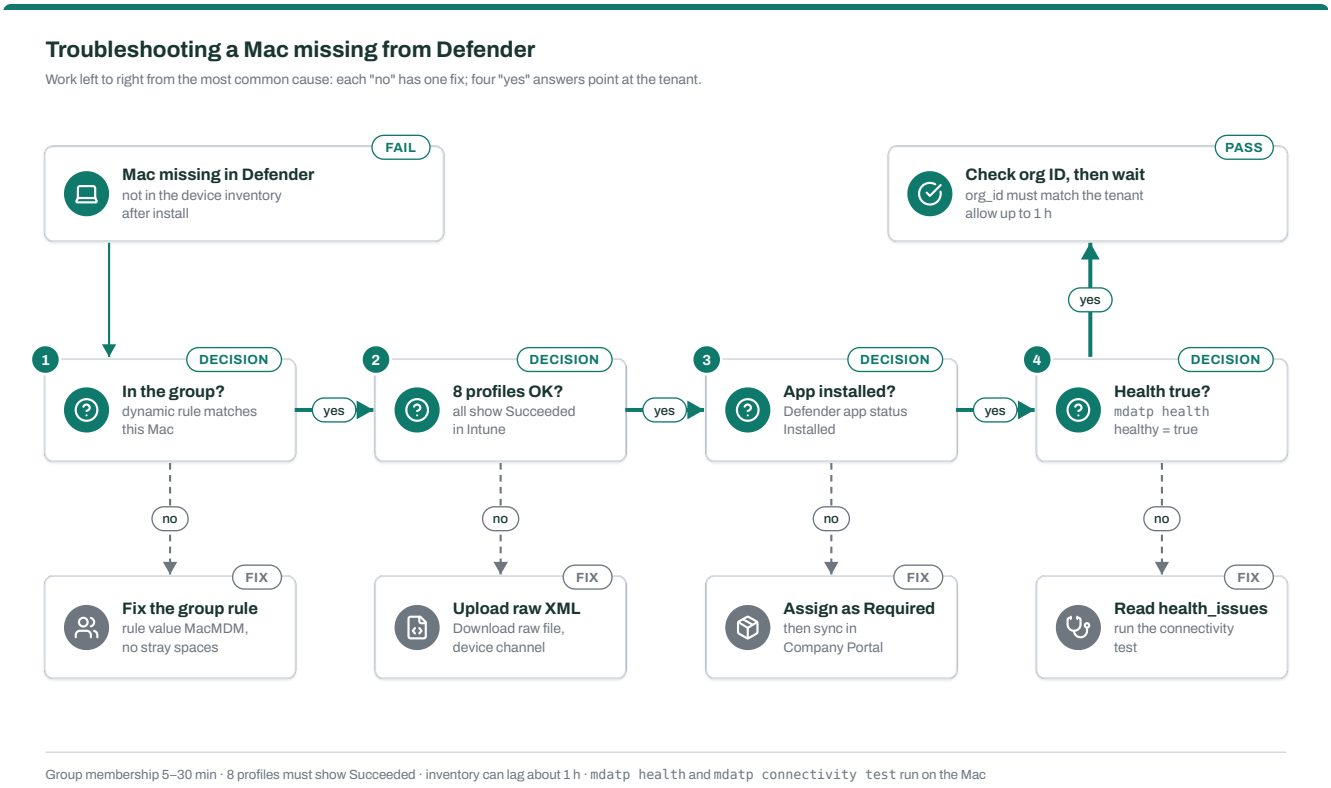


Figure 9. Most “missing device” cases are group, profile-file or assignment problems rather than agent faults, so rule those out before touching the Mac.

CHAPTER 7

Summary, Glossary and Review

7.1 Chapter summary

- MDM opens macOS privacy and security gates in advance, but only over a user-approved or ADE enrollment.
- Defender needs seven permission payloads, the onboarding payload and the app: device channel, dynamic group, app **Required**.
- Graph PowerShell makes the build repeatable; the Settings Catalog is the most maintainable form; local scripts and Jamf cover the rest.
- Prove success in Intune (delivered), the Defender portal (reporting) and `mdatp health` (healthy).

Check your understanding

1. Why must the system extension and Full Disk Access profiles arrive before the app?
2. The app is Installed in Intune but the Mac is not in Defender. What do you check first?
3. Why do these profiles use the device channel rather than the user channel?
4. What makes an enrollment "user-approved", and why does Defender depend on it?
5. When would you prefer the Settings Catalog, and what must you avoid when migrating?

7.2 Glossary

MDM	Mobile device management; Apple's protocol that lets a server install profiles and send commands to an enrolled device.
TCC / PPPC	macOS privacy permission system / the MDM payload that pre-approves it for named apps.
System extension	User-space replacement for kernel extensions, approved by team and bundle ID.
Device channel	MDM channel that installs payloads system-wide rather than for one user.
Onboarding package	Tenant-specific configuration that registers the Defender agent with your organization.
mdatp	Defender for Endpoint command-line tool on macOS and Linux.